In [2]:
```python
# Ex1a: Text Generation using N-gram Language Model

import nltk
from nltk.corpus import brown
from nltk.util import ngrams
from nltk.probability import ConditionalFreqDist


nltk.download('brown')
nltk.download('universal_tagset')

sentences = brown.sents()

tokens = [word.lower() for sent in sentences for word in sent]
print("Total number of tokens:", len(tokens))



# Bigram model
bigrams = ngrams(tokens, 2)
bigram_fd = ConditionalFreqDist()
for w1, w2 in bigrams:
    bigram_fd[w1][w2] += 1

# Trigram model
trigrams = ngrams(tokens, 3)
trigram_fd = ConditionalFreqDist()
for w1, w2, w3 in trigrams:
    trigram_fd[(w1, w2)][w3] += 1


def predict_bigram(word, top_n=5):
    word = word.lower()
    if word in bigram_fd:
        return bigram_fd[word].most_common(top_n)
    else:
        return []

def predict_trigram(word1, word2, top_n=5):
    key = (word1.lower(), word2.lower())
    if key in trigram_fd:
        return trigram_fd[key].most_common(top_n)
    else:
        return []

print("\n--- Bigram Prediction ---")
bigram_test_word = "the"
print(f"Input word: {bigram_test_word}")
print("Top-5 predictions:", predict_bigram(bigram_test_word))

print("\n--- Trigram Prediction ---")
trigram_test_words = ("the", "president")
print(f"Input words: {trigram_test_words}")
```

```
print("Top-5 predictions:", predict_trigram(*trigram_test_words))
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
Total number of tokens: 1161192

--- Bigram Prediction ---
Input word: the
Top-5 predictions: [('first', 662), ('same', 628), ('most', 417), ('other', 41
6), ('``', 405)]

--- Trigram Prediction ---
Input words: ('the', 'president')
Top-5 predictions: [('of', 16), ('and', 13), (',', 11), ('would', 8), ('.', 8)]
```

In [ ]: